

Anaconda Commands

Cheat Sheet

A quick reference for managing Conda environments, packages, channels, and reproducible Python environments.

Tip:

Conda manages both **packages** and **environments**. For reproducible projects, create a separate environment for each project instead of installing everything in the **base** environment.

1. Environment Management

```
conda create -name myenv
```

Create a new environment named myenv.

```
conda create -n myenv python=3.10
```

Create a new environment with Python 3.10 installed.

```
conda activate myenv
```

Activate the environment named myenv.

```
conda deactivate
```

Deactivate the currently active environment.

```
conda env list
```

Display all Conda environments available on the system.

```
conda env remove -n myenv
```

Delete the environment named myenv.

2. Package Management

```
conda install numpy
```

Install the latest compatible version of NumPy.

```
conda install -c conda-forge numpy
```

Install NumPy from the conda-forge channel.

```
conda install numpy=1.24
```

Install a specific version of NumPy.

```
conda update numpy
```

Update NumPy to a newer compatible version.

```
conda remove numpy
```

Remove NumPy from the current environment.

```
conda list
```

Display all packages installed in the current environment.

```
conda list numpy
```

Check whether NumPy is installed and display its installed version.

3. Export & Import Environments

```
conda env export > environment.yml
```

Export the current environment and its dependencies to an `environment.yml` file.

```
conda env create -f environment.yml
```

Create a new environment from an existing `environment.yml` file.

```
conda env update -f environment.yml --prune
```

Update an environment using an environment file and remove packages that are no longer required.

4. Search & Information

```
conda search numpy
```

Search available Conda repositories for NumPy.

```
conda info
```

Display information about the Conda installation and current configuration.

```
conda info --envs
```

Display all available Conda environments.

```
conda list
```

Display packages installed in the currently active environment.

5. Conda Configuration

```
conda config --show
```

Display the complete current Conda configuration.

```
conda config -set auto_activate_base false
```

Prevent the base environment from being activated automatically when opening a terminal.

```
conda config -add channels conda-forge
```

Add conda-forge to the list of Conda package channels.

```
conda config -show channels
```

Display the currently configured Conda channels.

6. Updating Conda

```
conda update conda
```

Update the Conda package manager itself.

```
conda update -all
```

Update all packages in the current environment to compatible versions.

7. Using Pip with Conda

```
pip install package_name
```

Install a package using pip inside the currently active environment.

```
python -m pip install package_name
```

Install a package using the pip associated with the currently active Python interpreter.

```
conda run -n myenv pip install package_name
```

Run pip inside myenv without explicitly activating the environment.

Best Practice:

Install as many packages as possible using Conda first.

If a package is unavailable through Conda, install it with pip **after** the Conda packages have been installed.

This reduces the chance of dependency conflicts.

8. Useful Shortcuts

```
conda create -n myenv python=3.12 numpy pandas matplotlib
```

Create an environment with Python and several packages using a single command.

```
conda activate myenv && python -version
```

Activate an environment and immediately verify its Python version.

```
conda clean -all
```

Remove unused package caches, index caches, tarballs, and other temporary Conda files.

9. Typical Workflow

Creating a New Python Project

```
# Create a new environment
conda create -n myproject python=3.12

# Activate the environment
conda activate myproject

# Install packages
conda install numpy pandas matplotlib

# Install a pip-only package if necessary
python -m pip install package_name

# Export the environment
conda env export > environment.yml

# Leave the environment
conda deactivate
```

Recreating a Project Environment

```
# Create environment from YAML file
conda env create -f environment.yml

# Activate the environment
conda activate myproject

# Check installed packages
conda list
```